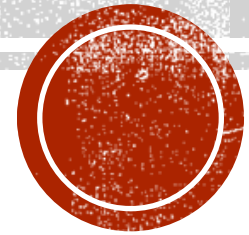


GPU ARCHITECTURE & CUDA PROGRAMMING

CAT Team
aca2025@eclectx.org



AGENDA

GPU Architecture

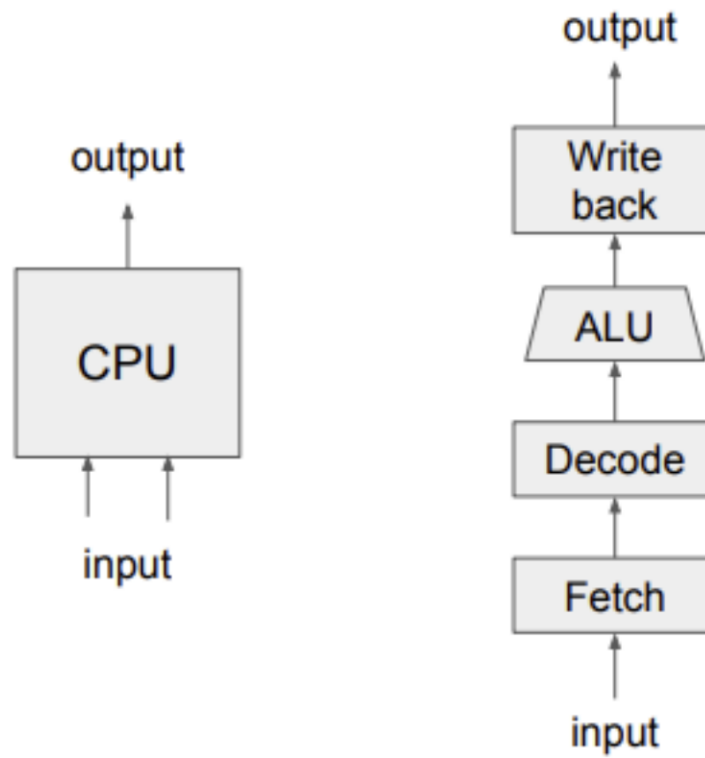
CUDA Programming Model

Mapping CUDA to GPU Hardware

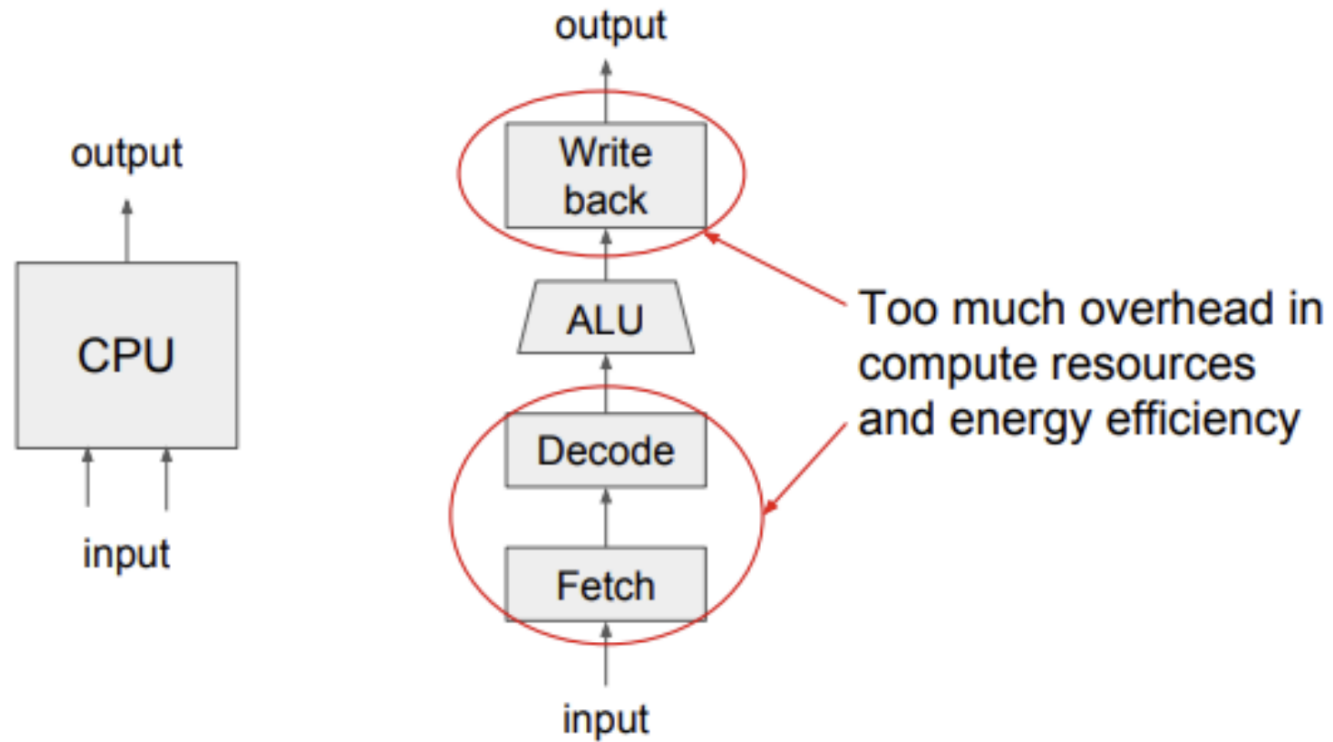
Lab Agenda



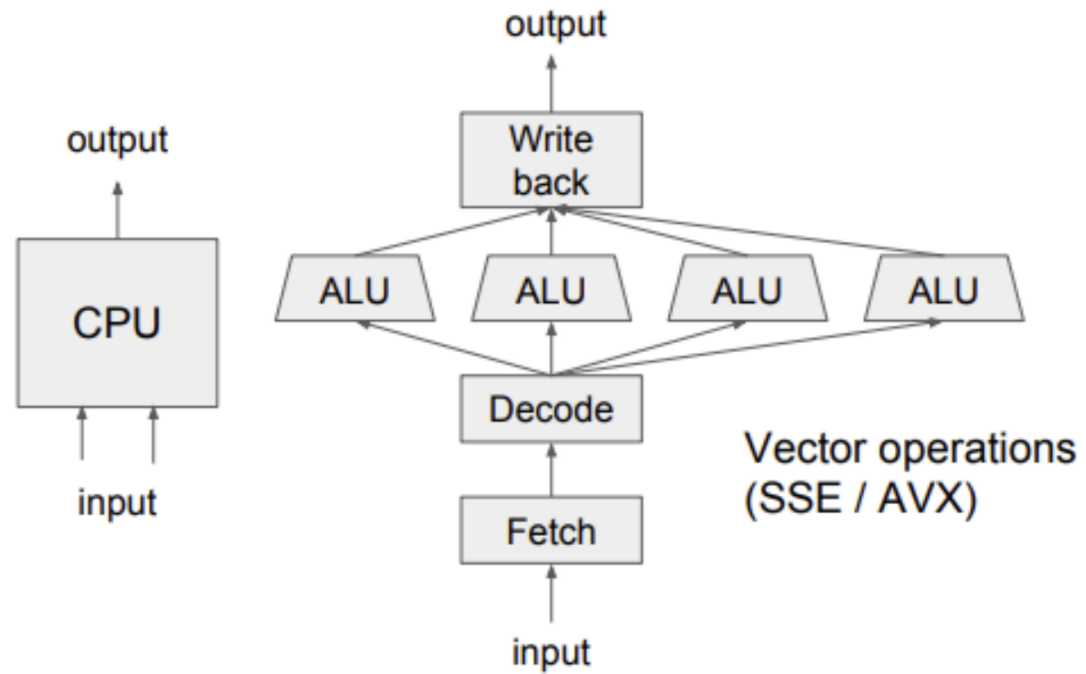
CPU PIPELINE



CPU PIPELINE

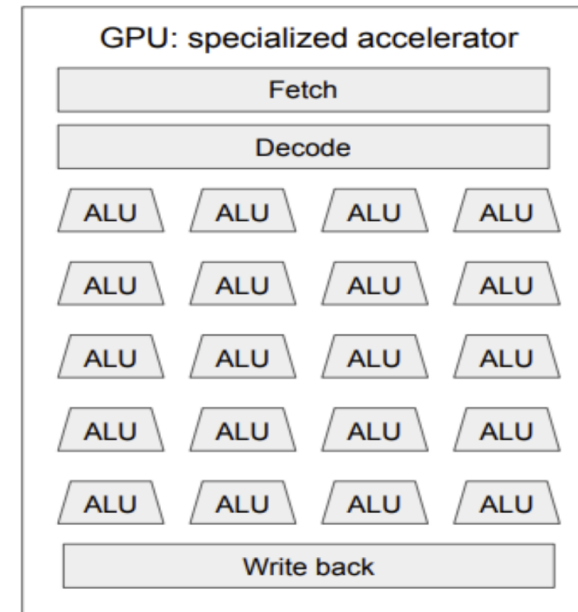


CPU PIPELINE



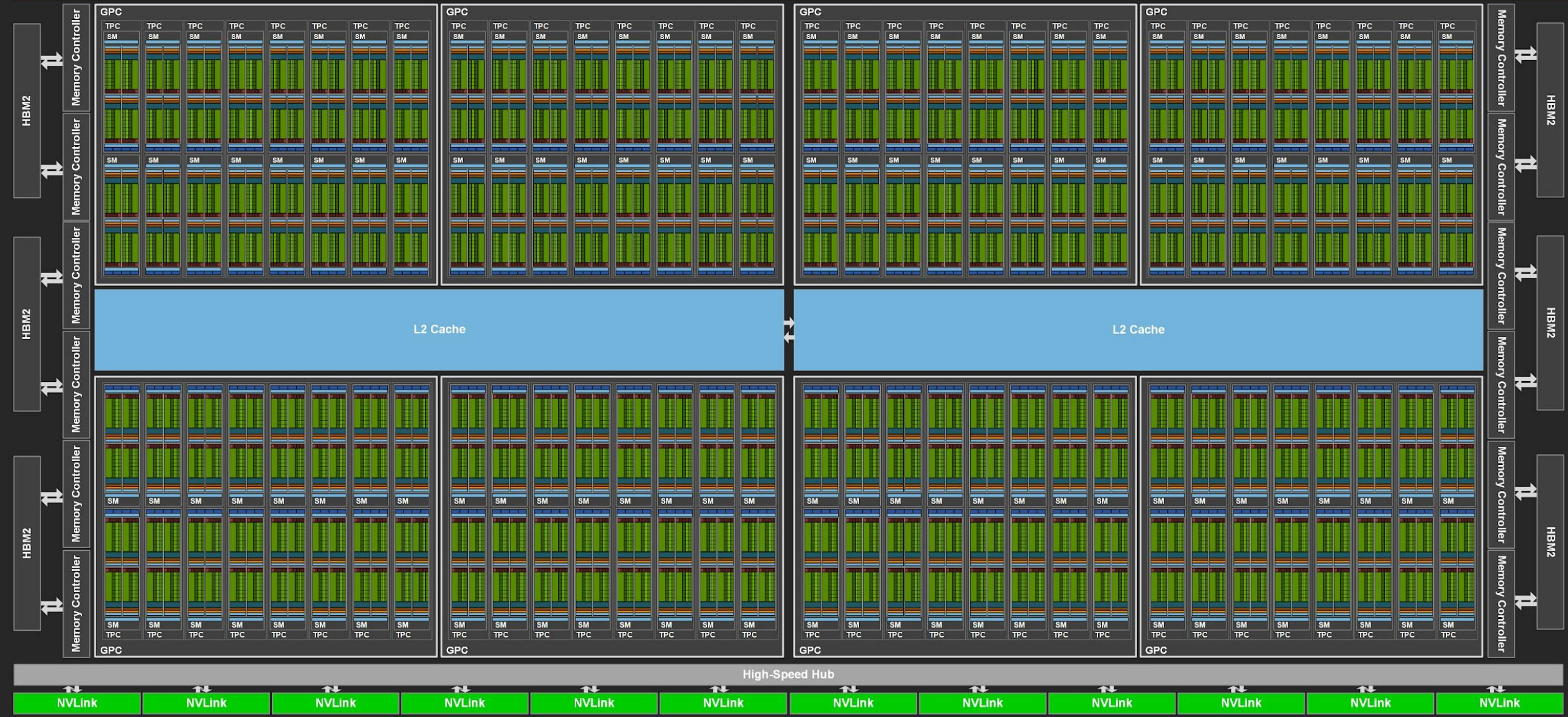
GPU PIPELINE

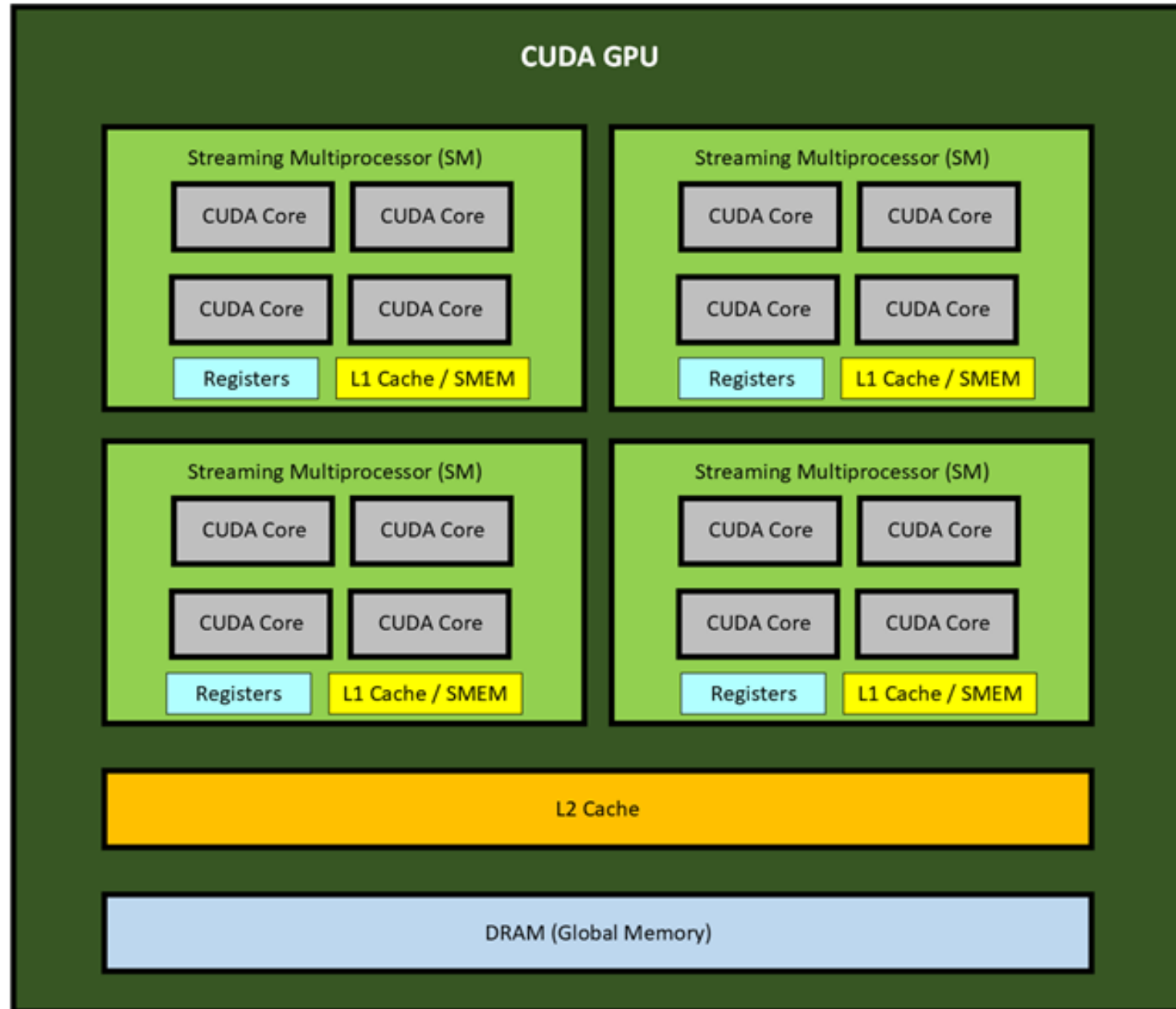
SIMD: single instruction multiple data



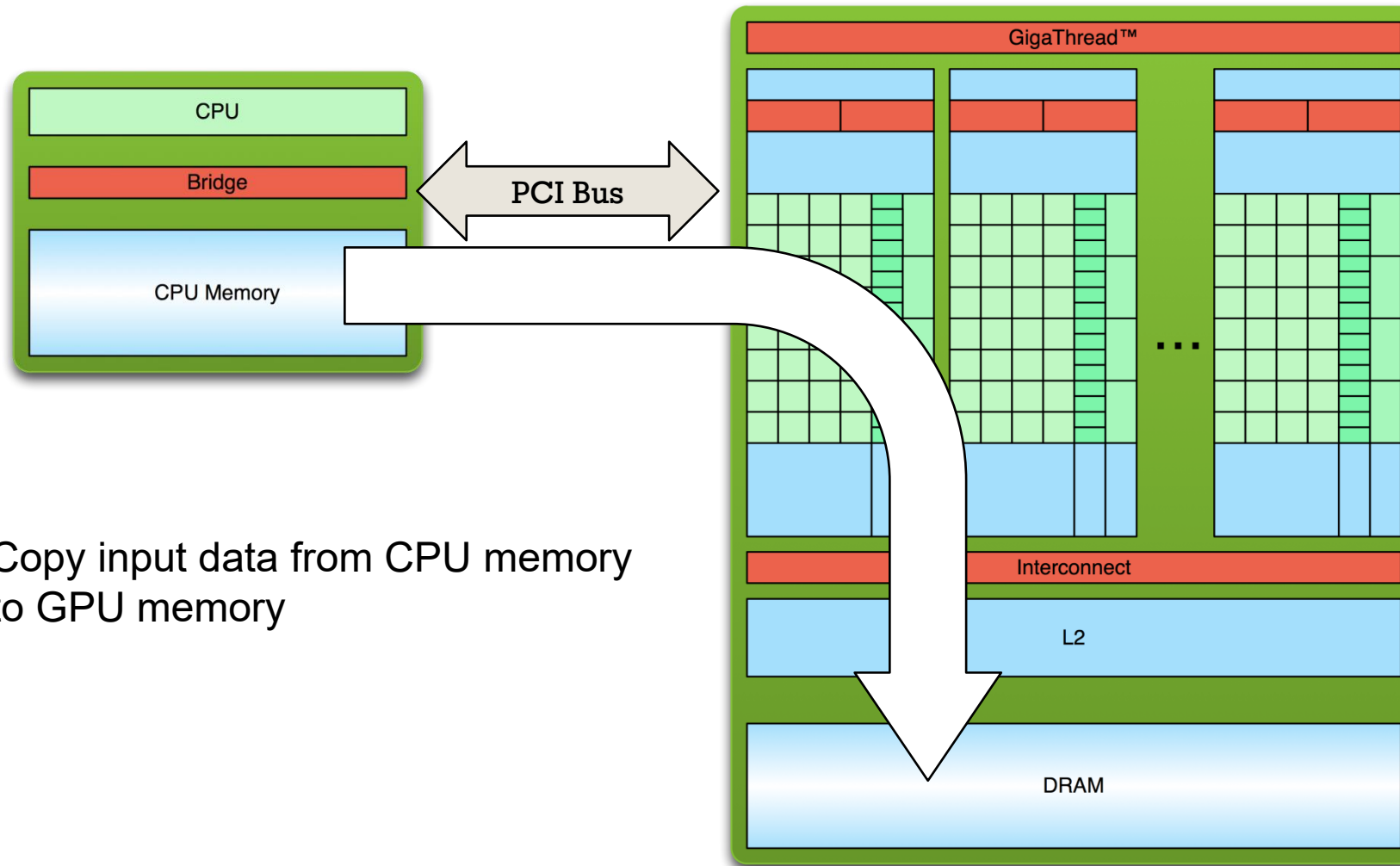
PCI Express 4.0 Host Interface

GigaThread Engine with MIG Control





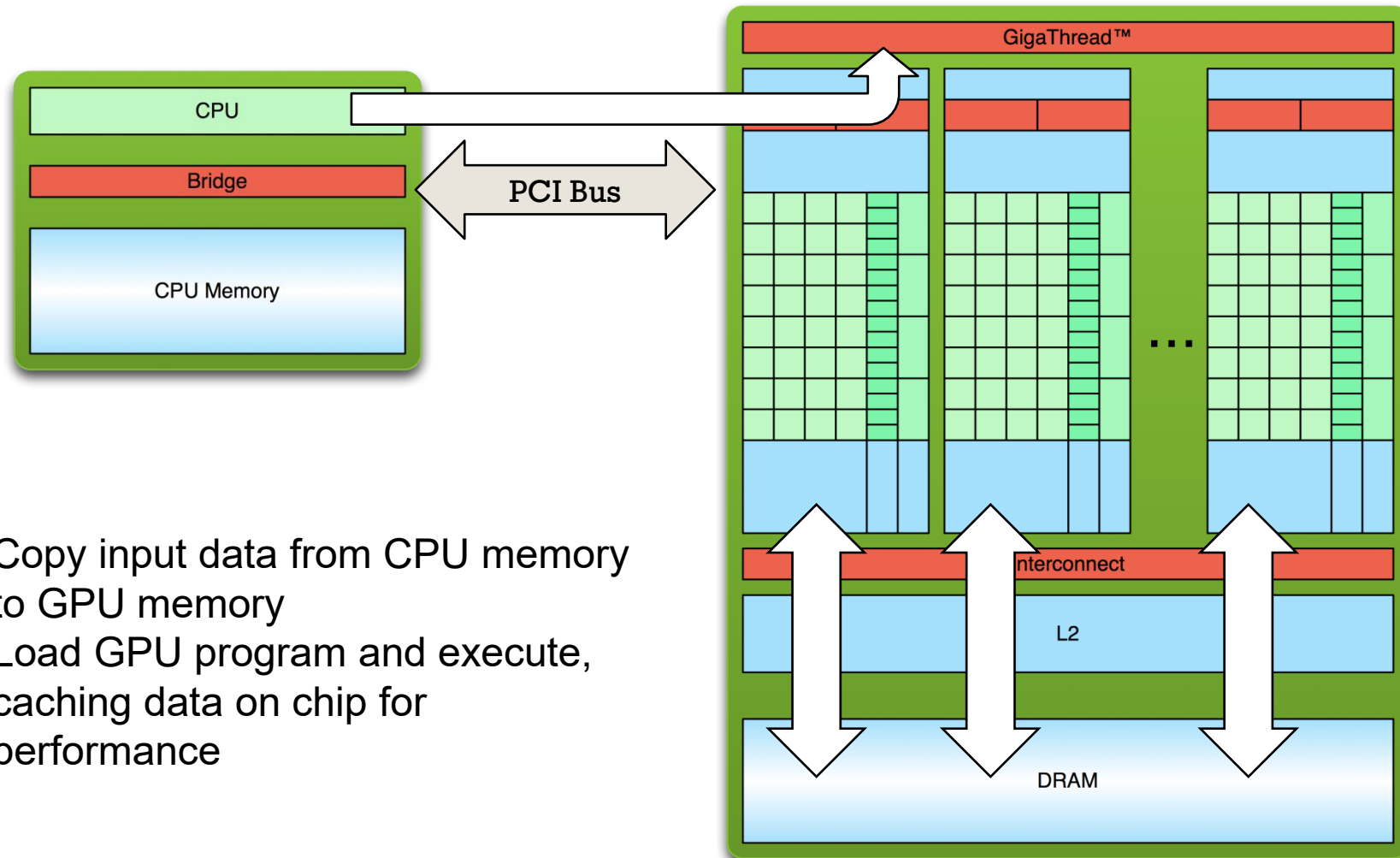
SIMPLE PROCESSING FLOW



1. Copy input data from CPU memory to GPU memory



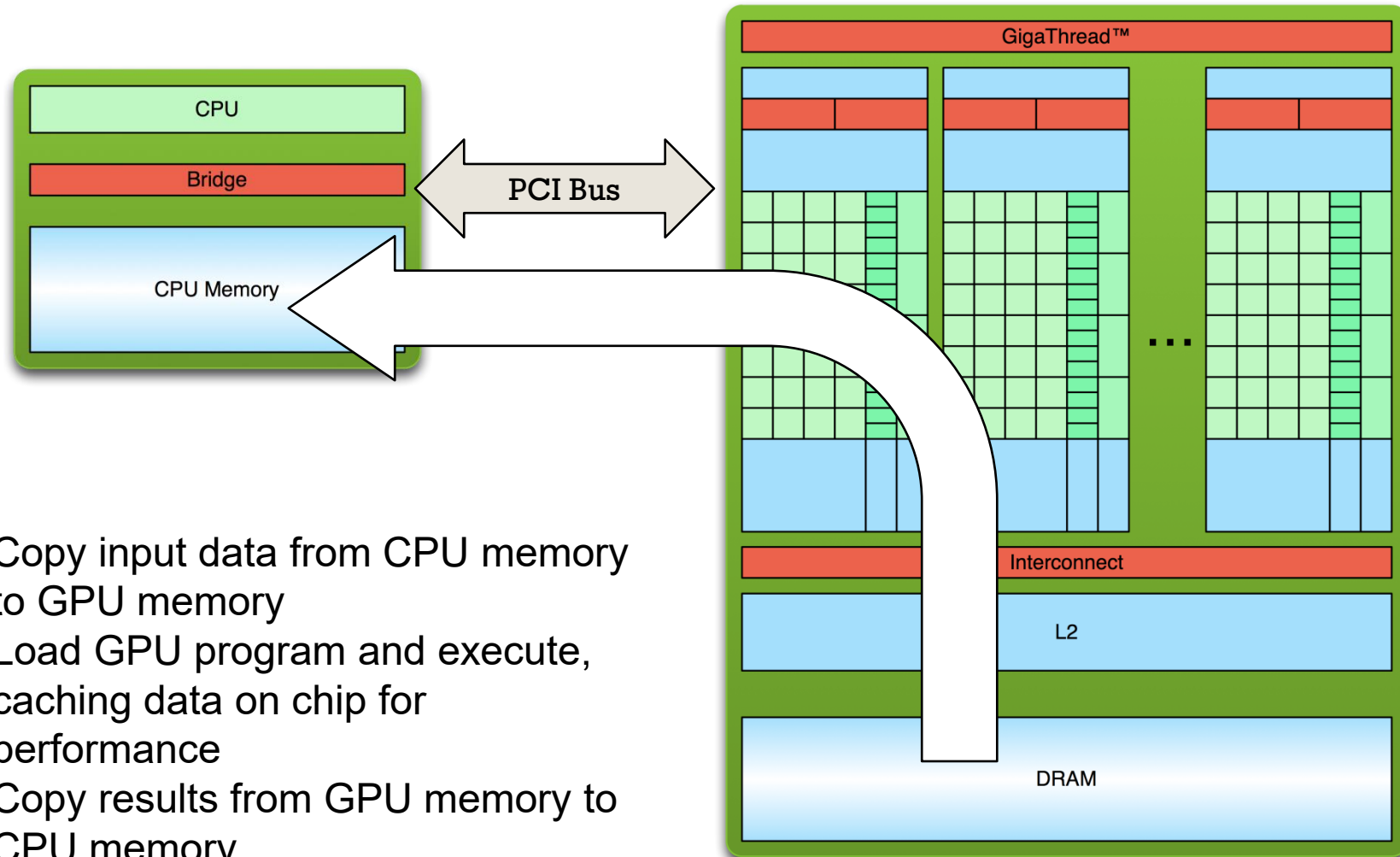
SIMPLE PROCESSING FLOW



1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance



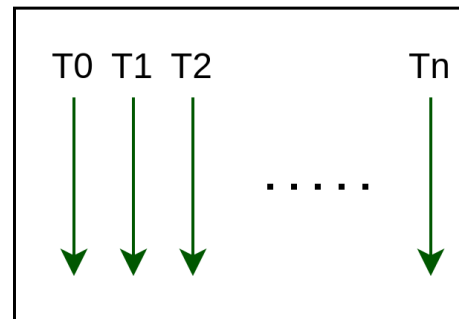
SIMPLE PROCESSING FLOW



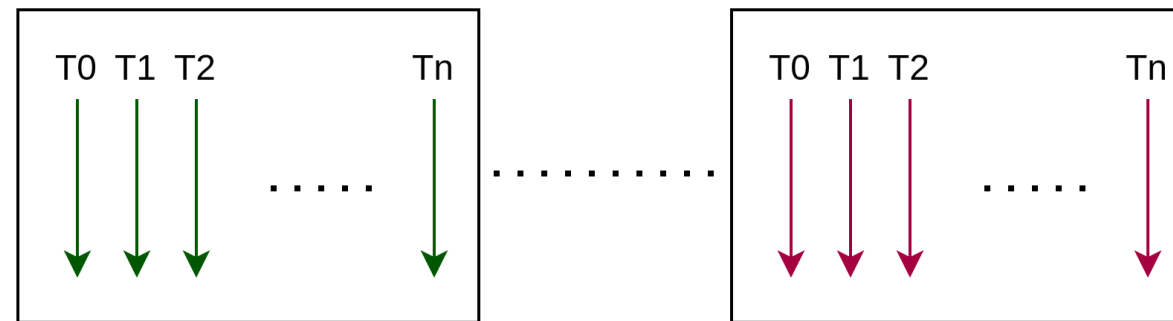
Thread



Thread
Block



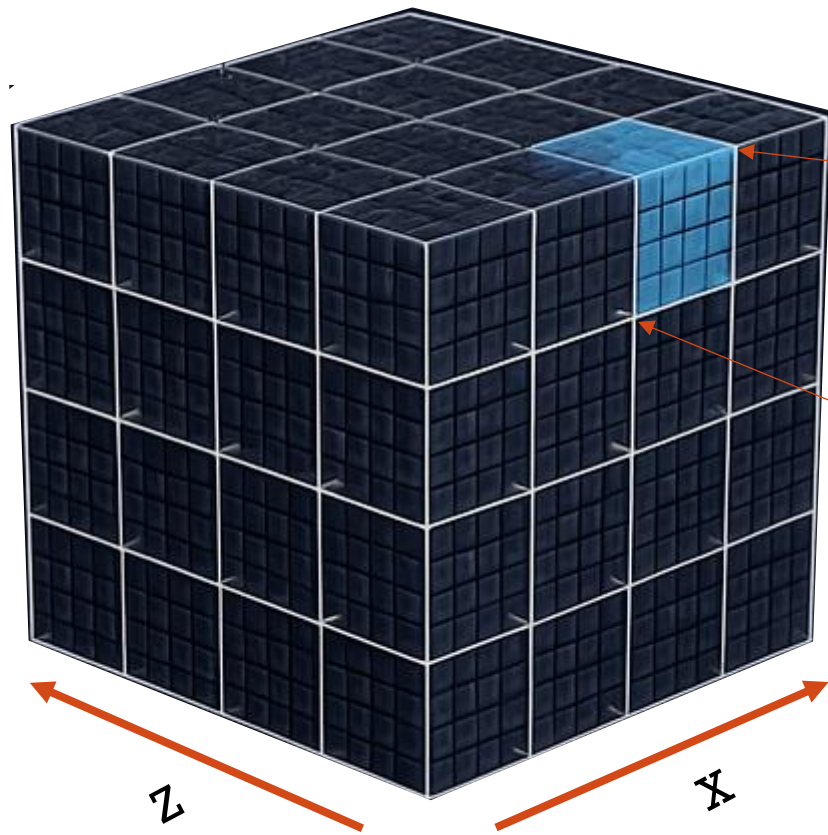
Kernel
Grid



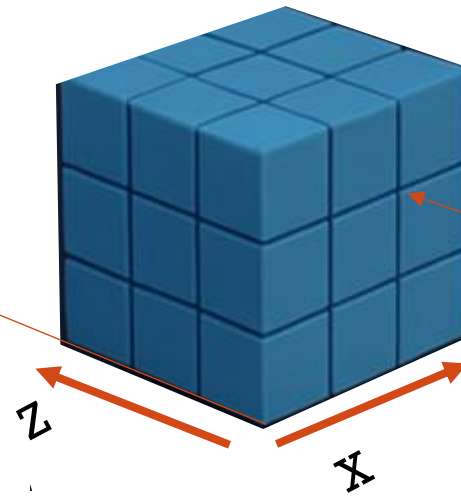
B0

Bm

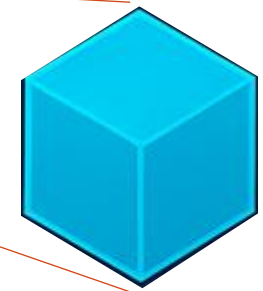




Grid of Blocks

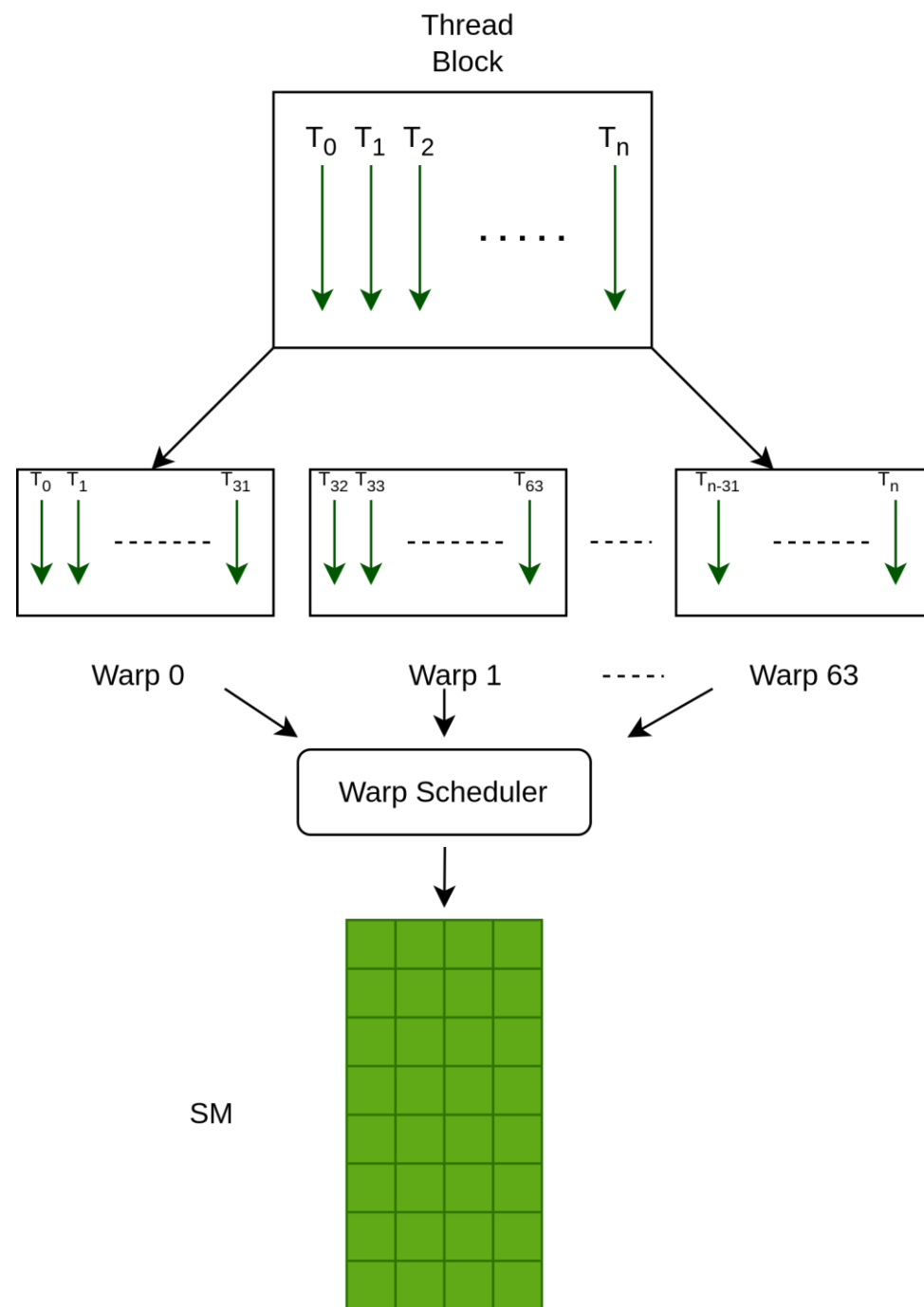


Block of Threads



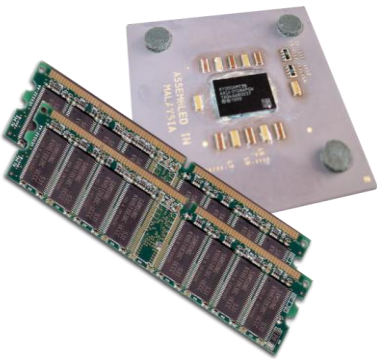
Thread





HETEROGENEOUS COMPUTING

- Terminology:
 - *Host* The CPU and its memory (host memory)
 - *Device* The GPU and its memory (device memory)



Host



Device



Hello World!

```
int main(void) {  
    printf("Hello World!\n");  
    return 0;  
}
```

Standard C that runs on the host

NVIDIA compiler (nvcc) can be used to compile programs with no *device* code



```
!nvcc -o Helloworld_Host Helloworld_Host.cu.cu  
!./Helloworld  
  
Helloworld!
```



Hello World!

```
__global__ void mykernel() {  
}
```

```
int main(void) {  
    mykernel<<<1,1>>>();  
    printf("Hello World!\n");  
    return 0;  
}
```

- mykernel() is called and it does nothing!



Hello World!

```
#include <cuda_runtime.h>
```

```
#include <stdio.h>
```

```
__global__ void mykernel() {  
  
}
```

```
int main(void) {
```

```
    mykernel<<<1,1>>>>();
```

```
    printf("Hello World!\n");
```

```
    return 0;
```

```
}
```

- mykernel() is called.



Hello World!

```
#include <cuda_runtime.h>
#include <stdio.h>

__global__ void mykernel() {
    int idx = threadIdx.x;
    printf("\nMy thread id is %d",idx);
}

int main(void) {
    mykernel<<<1,1>>>();
    printf("Hello World!\n");
    return 0;
}
```

- mykernel() is called and it prints its thread id.



Helloworld example

Vector Addition

Matrix Multiplication and optimization



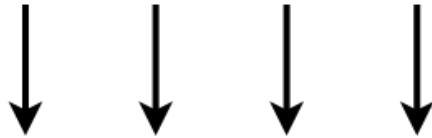
VECTOR ADDITION

A



+

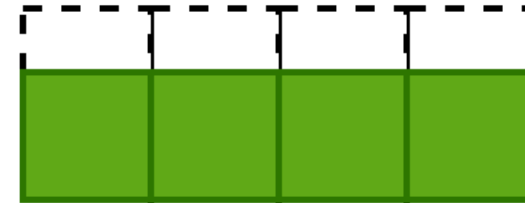
B



SUM

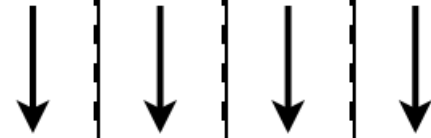


A



+

B



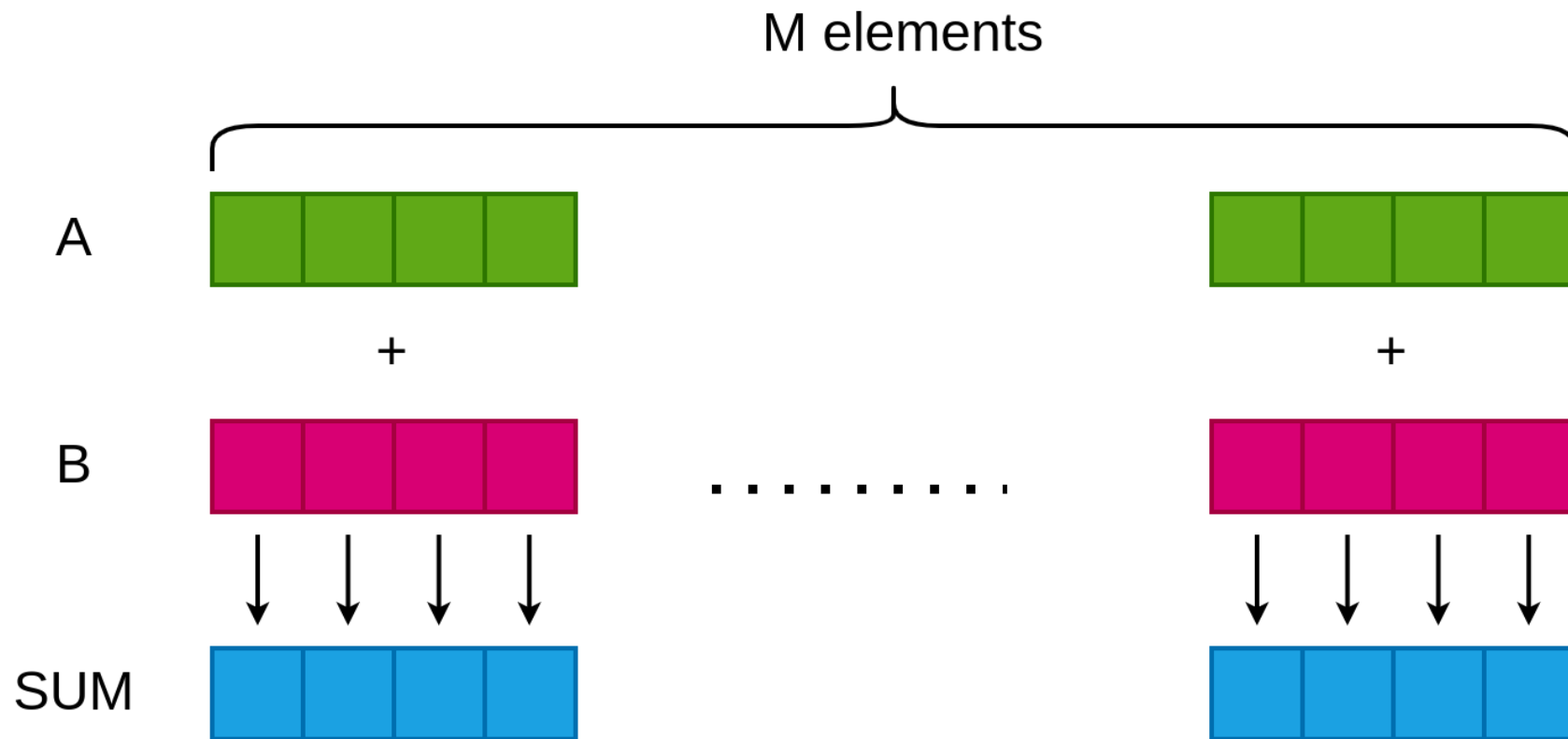
SUM



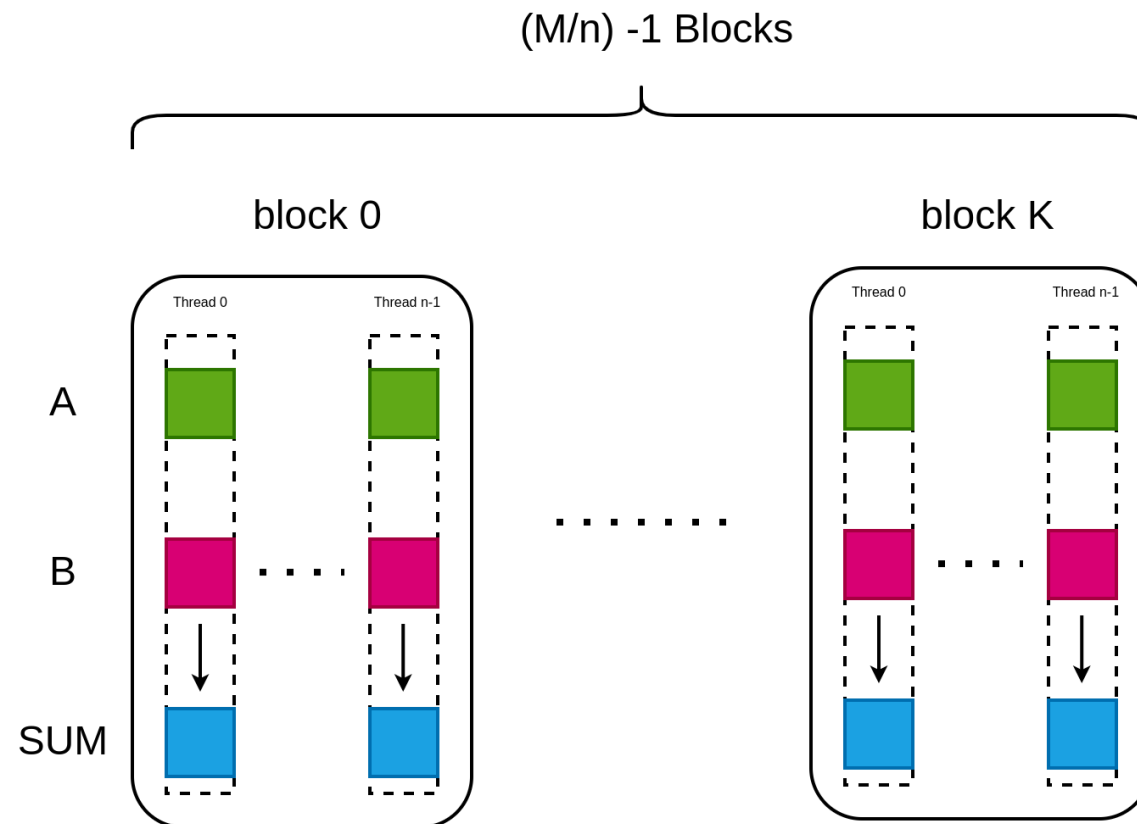
Thread 0 Thread 1 Thread 2 Thread 3



BIG VECTOR ADDITION



BIG VECTOR ADDITION WITH MULTIPLE BLOCKS



MATRIX MULTIPLY

Matrix A

Matrix B

Matrix C



MATRIX MULTIPLY

Matrix A

Matrix B

Matrix C

